SCHOOL OF COMPUTING AND DATA SCIENCES
FLAME UNIVERSITY
PUNE - 412115, MAHARASHTRA, INDIA

A Staged TCP/IP-Inspired Transport Protocol Implementation over UDP in C

A Graduation Project

Submitted by

PRATIK PATIL

For the award of the degree

Of

BSC. COMPUTER SCIENCE AND DESIGN

April 2026

THESIS CERTIFICATE

This is to undertake that the Graduation Project titled **A STAGED TCP/IP-INSPIRED TRANSPORT PROTOCOL IMPLEMENTATION OVER UDP IN C**, submitted by me to the School of Computing and Data Sciences, FLAME University, for the award of **BSc. Computer Science and Design**, is a bonafide record of the research work done by me under the supervision of **Dr. Aamod Sane**. The contents of this Graduation Project, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

In order to effectively convey the ideas presented in this Graduation Project, all protocol diagrams, comparative figures, and analytical tables in the main report were created by the author specifically for this submission.

Standard technical references cited in the bibliography are used only for academic context and conceptual grounding. No copyrighted third-party figure is reproduced in the main body of the report.

Pune - 412115, Maharashtra, India **Pratik Patil**

Date: April 2026

Dr. Aamod Sane
School of Computing and Data Sciences
FLAME University

© 2026 FLAME University

ACKNOWLEDGEMENTS

This graduation project would not have been possible without the help and support of many people and organizations. I want to thank everyone who helped me with this work from the bottom of my heart.

First and foremost, I would like to thank **Dr. Aamod Sane**, my research advisor, for all of his help, advice, and support during this project. His guidance helped me understand transport-layer design better and made this thesis more technically sound and easier to read.

I want to thank the **School of Computing and Data Sciences at FLAME University** for giving me a good academic setting, helpful learning materials, and institutional support that I needed to do this work.

I also want to thank my friends and coworkers for their helpful conversations, suggestions for how to implement things, and moral support while I was working on the project, fixing bugs, and writing the report.

Lastly, I want to thank my family from the bottom of my heart for being patient, understanding, and always pushing me to do my best. Their help gave me the confidence and focus I needed to finish this graduation project.

ABSTRACT

KEYWORDS TCP/IP; UDP sockets; reliability; checksum; sliding window; selective repeat; adaptive timeout; congestion control; fast retransmit; fast recovery

This report describes how to design and build a TCP/IP-inspired communication stack in C that works over UDP sockets. There are twenty-five sender-receiver pairs in the project, from level 0 to level 24. It shows a clear progression from simple bit transfer to more complex transport behavior that is similar to real TCP principles. The first levels are all about encoding and decoding messages. Middle levels add checking for parity, acknowledgments, retries based on timeouts, and reliability through stop-and-wait. Later levels add things like setting up and tearing down connections, packet checksums, sequence and acknowledgment numbers, pipelining, sliding windows, chunk-based payload transfer, out-of-order buffering, and adaptive retransmission timeout (RTO) estimation.

The full project also includes the expanded web visualizer, which shows versions 0 to 24. The code, finite state machine, and UML timeline are all shown together in that interface. This makes it easier to learn about the protocol progression as a whole rather than as separate source files.

A key goal of this project is to learn networking from the scratch through a textbook-first approach. The conceptual basis follows to established references, including the top-down networking approach by Kurose and Ross and the implementation viewpoint of Stevens. AI tools were only used in a responsible way to help with idea exploration, clarification, and debugging. The final steps of implementation, coding, and validation were done by hand, without copying any code.

The final levels implement congestion control ideas such as slow start, congestion avoidance, and, in the most advanced version, fast retransmit and fast recovery based on duplicate acknowledgements. This project is an educational simplification of TCP and uses UDP as the underlying transport. It captures many practical protocol engineering concepts, such as correctness, efficiency, robustness to packet loss and reordering, and maintainable protocol state management.

The educational value is very important. The versioned C code, textbook references, and synchronized visualizer all work together to make TCP-like behavior easier for new students to understand. These students often find transport protocols and congestion control hard to understand at first.

The report talks about the architecture, algorithms, implementation choices, version-by-version progress, and limitations. It also has diagrams, tables that compare things, and analysis based on formulas for throughput and timeout logic. The result is a full learning tool that connects networking theory with real-world systems programming.

CONTENTS

		Page
ACKNOWLEDGEMENTS		**i**
ABSTRACT		**ii**
LIST OF TABLES		**vii**
LIST OF FIGURES		**viii**
CHAPTER 1	**INTRODUCTION**	**1**
1.1	Problem Statement	2
1.2	Project Objectives	3
1.3	System Context and Setup	4
1.4	Architecture Overview	4
1.5	Version Progression Strategy	5
1.6	Version-Wise Reference Backbone	5
1.7	Educational Value	7
1.8	Scope and Boundaries	8
1.9	Research Questions	8
CHAPTER 2	**PROTOCOL DESIGN AND IMPLEMENTATION**	**9**
2.1	Global Design Principles	9
2.2	Version Family A: Bit and Byte Transfer (v0–v2)	10
2.2.1	Version 0: Single-Bit UDP Demo	10
2.2.2	Version 1: Character as 8 Independent Bits	11
2.2.3	Version 2: Full String by Bit Stream	12
2.3	Version Family B: Error Detection and Stop-and-Wait (v3–v7)	13
2.3.1	Version 3: Parity Detection	13
2.3.2	Version 4: Sequence Numbers	14
2.3.3	Version 5: Stop-and-Wait ARQ with ACK/NAK	15
2.3.4	Version 6: Timeout and Retries	16
2.3.5	Version 7: Struct Based Packetization	17
2.4	Version Family C: TCP-like Framing and Pipelining (v8–v10)	18
2.4.1	Version 8: Handshake, Flags, and Connection Lifecycle	18
2.4.2	Version 9: Sliding Window (Go-Back-N Style)	19
2.4.3	Version 10: Chunk-Based Payloads	20
2.5	Version Family D: Selective Repeat and Adaptive Control (v11–v14)	21
2.5.1	Version 11: Selective-Repeat Like Buffering	21
2.5.2	Version 12: Adaptive RTO Estimation	22
2.5.3	Version 13: Congestion Window and Slow Start	23
2.5.4	Version 14: Fast Retransmit and Fast Recovery	24

2.6 Version Family E: Adaptive and Reno Like Extensions (v15–v24) . . . 25
2.6.1 Version 15: RTT-Smoothed Timeout Refinement 26
2.6.2 Version 16: Adaptive Window Discipline 27
2.6.3 Version 17: Timestamp Option Integration 28
2.6.4 Version 18: SACK-Inspired Receiver Reporting 29
2.6.5 Version 19: Congestion Avoidance Stabilization 30
2.6.6 Version 20: Loss-Recovery Coupled with Option Feedback 31
2.6.7 Version 21: Sender State Consolidation 32
2.6.8 Version 22: Receiver Reassembly and Delivery Hardening 33
2.6.9 Version 23: End-to-End Robustness Tuning 34
2.6.10 Version 24: Final Integrated Transport Stage 35
2.6.11 Family E Summary 36
2.7 Chapter Summary 36

CHAPTER 3 ALGORITHMIC AND QUANTITATIVE ANALYSIS 37

3.1 Bit Encoding and Message Cost 37
3.2 Parity-Based Error Detection 37
3.3 Stop-and-Wait Service Time 38
3.4 Checksum in TCP-like Versions 38
3.5 Sliding Window Efficiency 39
3.6 Chunking Gain 39
3.7 Adaptive RTO Estimation 40
3.8 Congestion Window Dynamics 40
3.8.1 Slow Start 40
3.8.2 Congestion Avoidance 40
3.8.3 Loss Response 41
3.9 Latency Components 41
3.10 Reliability Metrics 42
3.11 A Practical Theorem-like Observation 42
3.12 Interpretation for This Project 43

CHAPTER 4 ILLUSTRATIONS, COMPARATIVE ANALYSIS, AND FINAL OUTCOMES 44

4.1 End-to-End Protocol Lifecycle 44
4.2 Version Capability Layers 45
4.3 Reliability Progress Chart 46
4.4 Throughput-Oriented Improvement Chart 47
4.5 Comparative Feature Matrix 48
4.6 Loss-Recovery Behavior Illustration 49
4.7 Key Findings 49
4.8 Limitations 49
4.9 Conclusion 50

APPENDIX A EXTENDED TECHNICAL APPENDIX 51

A.1 Appendix Purpose 51

A.2 Complete Build and Run Workflow 51
A.2.1 Compile 51
A.2.2 Execute 51
A.2.3 Operational Notes 52
A.3 Debug Checklist Used in This Project 52
A.4 Detailed Behavior: Sender Core Loop 53
A.4.1 State Variables 53
A.5 Detailed Behavior: Receiver Core Loop 54
A.5.1 Receiver Buffering in Selective-Repeat Style 54
A.6 Version-by-Version Pitfall Notes 54
A.6.1 v0–v2 Pitfalls 54
A.6.2 v3–v7 Pitfalls 54
A.6.3 v8–v10 Pitfalls 55
A.6.4 v11–v14 Pitfalls 55
A.7 Experiment Design Template 55
A.8 Sample Observation Narrative 56
A.9 Illustrative State Machine View 56
A.10 Protocol Trace Formatting Recommendation 56
A.11 Suggested Quantitative Extensions 57
A.12 Security and Reliability Perspective 57
A.13 Code Organization Recommendations 58
A.14 Reproducibility Notes 58
A.15 Expanded Testing Scenarios and Interpretation 58
A.16 Evaluation Rubric for Academic Assessment 59
A.17 Final Reflection 60
A.18 Appendix Summary 60

BIBLIOGRAPHY **61**

LIST OF TABLES

Table	Caption	Page
1.1	Main references used for each protocol family.	6
4.1	Feature comparison across key protocol milestones.	48
A.1	Standard experiment template used for each protocol version.	55
A.2	Suggested assessment rubric for implementation quality.	59

LIST OF FIGURES

Figure	Caption	Page
1.1	Basic sender and receiver flow over UDP.	4
2.1	Version 0: one bit sent over UDP.	10
2.2	Version 1 architecture: one character transmitted as eight UDP bit datagrams and reconstructed at receiver side.	11
2.3	Version 2 architecture: full string transmitted as a bit stream with NULL-terminated completion.	12
2.4	Version 3 architecture: parity based corruption detection over UDP payload transfer.	13
2.5	Version 4 architecture: sequence numbered UDP units for order visibility and reliability groundwork.	14
2.6	Version 5 architecture: stop-and-wait ARQ with ACK/NAK feedback path.	15
2.7	Version 6 architecture: timeout driven retransmission with duplicate safe receiver handling.	16
2.8	Version 7 architecture: struct based packetization with unified sender and receiver parsing flow.	17
2.9	Version 8 architecture: handshake flags, lifecycle control, and checksum protected data flow.	18
2.10	Version 9 architecture: sliding window pipelining with cumulative ACK and timeout based retransmission.	19
2.11	Version 10 architecture: chunk based payload transfer with in-window buffering and ordered reassembly.	20
2.12	Version 11 architecture: selective-repeat like buffering with out-of-order storage and in-order delivery.	21
2.13	Version 12 architecture: RTT sampling with adaptive RTO control for variable delay conditions.	22
2.14	Version 13 architecture: congestion window growth, ACK clocking, and loss triggered rate reduction.	23
2.15	Version 14 architecture: duplicate ACK loss inference with fast retransmit and fast recovery control.	24
2.16	Version 15 architecture: smoothed RTT driven retransmission timeout refinement.	26
2.17	Version 16 architecture: timeout aware adaptive window control.	27
2.18	Version 17 architecture: timestamp insertion and echo aware feedback loop.	28
2.19	Version 18 architecture: SACK-inspired gap reporting and selective resend planning.	29
2.20	Version 19 architecture: stabilized congestion avoidance behavior with ACK clocked feedback.	30
2.21	Version 20 architecture: fast recovery guided by richer feedback signals.	31

2.22 Version 21 architecture: consolidated sender control for stable decision flow. 32
2.23 Version 22 architecture: hardened receiver reassembly and contiguous delivery control. 33
2.24 Version 23 architecture: coordinated end-to-end robustness tuning under mixed failures. 34
2.25 Version 24 architecture: final integrated transport design across reliability, timing, and recovery. 35

4.1 Full session flow for the advanced versions. 44
4.2 How the feature set grows across version families. 45
4.3 Reliability trend as the protocol grows. 46
4.4 Throughput trend across the staged versions. 47
4.5 Fast loss recovery in the advanced versions. 49

A.1 Simplified sender-side operational state flow in advanced versions. . . . 56
A.2 Recommended structured protocol-trace fields for automated analysis. . 56

CHAPTER 1

INTRODUCTION

A major issue with computer networks is that they are not always reliable. At first, sending data from one computer to another may seem easy. In practice, real communication has to deal with packet loss, duplication, corruption, delay variation, and delivery that is not in order. TCP and other transport protocols were made to fix these problems. However, the behavior of TCP can feel abstract when studied only through theory.

This project adddresses that learning gap by gradually adding transport protocol features to UDP sockets in C. The project doesn't use the operating system's TCP stack directly. Instead, it builds many TCP-like ideas into the application code. This method shows every design choice. The logic for sending and receiving, managing sequences, handling timeouts, and responding to congestion are all clearly defined and followed.

There are twenty-five sender-receiver versions in the full work, from level 0 to level 24. The first levels use bit transmission and byte reconstruction. The middle levels add error detection and reliability that stops and waits. The advanced levels have ideas for buffering that are like TCP Reno behavior, such as handshake, checksum, sliding window, selective-repeat style buffering, adaptive timeout estimation, timestamp, and SACK-like ideas.

1.1 PROBLEM STATEMENT

The project aims to answer one practical question:

> How can we incrementally construct a reliable and adaptive transport protocol over unreliable UDP datagrams while keeping the implementation understandable for learning and experimentation, while also using AI tools responsibly for idea support (not code copying) and validating every design step through standard networking textbooks?

UDP only guarantees best-effort delivery of datagrams. It doesn't make sure that packets arrive, that they are in the right order, that duplicates are removed, that congestion is controlled, or that the connection state is correct. So, the application layer has to use these features if they are needed. This project shows how to do that step by step. The learning method in this work is book-first and concept-first: AI tools were only used to talk about other options, help with debugging, and come up with pseudocode ideas. The final logic, code writing, and verification were all done by hand.

The conceptual path follows a top-down networking approach in the spirit of widely used teaching references, especially Kurose and Ross for protocol layering and transport intuition, Stevens for implementation-level thinking about TCP/IP behavior, and Tanenbaum and Wetherall for broader networking foundations Kurose and Ross (2021); Stevens and Wright (1995); Tanenbaum and Wetherall (2011); Postel (1981); Paxson *et al.* (2011); Allman *et al.* (2009); Mathis *et al.* (1996); Borman *et al.* (2014); Postel (1980); Hall (2024).

1.2 PROJECT OBJECTIVES

The major objectives of the project are listed below.

1. Build a complete progression from raw transmission to transport control.

2. Keep each version self-contained, so learners can compare behavior across levels.

3. Follow a responsible learning process: study textbook concepts first, then use AI tools for idea exploration, and finally implement and test independently without copying generated code.

4. Implement reliability features step by step: parity, ACK/NAK, timeout retries, sequence control, and retransmission logic.

5. Move from one-byte packets to chunk-based payload transfer to improve throughput.

6. Introduce adaptive timing and congestion-aware sending to mimic practical transport design.

7. Document implementation, assumptions, limitations, and future improvements in a technical report.

1.3 SYSTEM CONTEXT AND SETUP

All implementations use IPv4 UDP sockets (`AF_INET` with `SOCK_DGRAM`) on port 9090. The sender transmits to localhost (`127.0.0.1`), and the receiver binds to `INADDR_ANY`. Typical message size in this project is bounded to around 99 characters.

The same high-level execution pattern is followed in all versions:

1. Start the receiver first.
2. Then start the sender.
3. Type the message in the sender terminal.
4. Watch the transfer, ACKs, retries, and finish.

1.4 ARCHITECTURE OVERVIEW

Figure 1.1 shows the conceptual architecture. Even though the implementation is local (single host), the control logic is equivalent to a networked deployment.

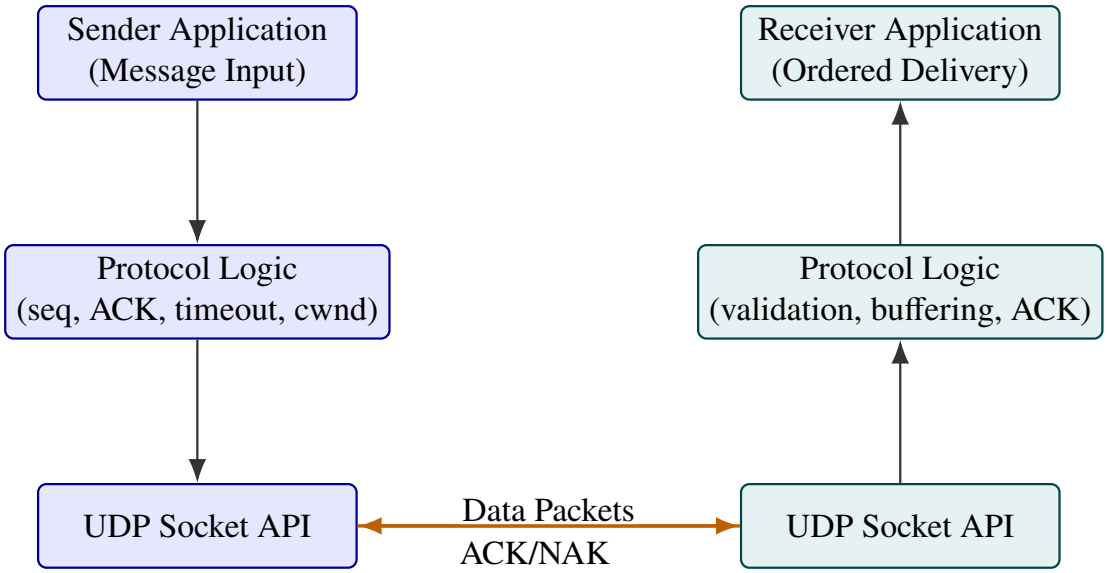

Figure 1.1: Basic sender and receiver flow over UDP.

1.5 VERSION PROGRESSION STRATEGY

The project does not attempt to implement all features at once. Instead, it follows a layered progression model:

1. **Foundation phase (v0–v2):** bit and byte transfer.
2. **Reliability phase (v3–v7):** parity, sequence tracking, stop-and-wait, timeout retries, packet structure.
3. **Transport phase (v8–v14):** handshake, checksum, connection close, sliding window, chunking.
4. **Adaptive and Reno-like phase (v15–v24):** adaptive RTO, timestamps, SACK-style hints, congestion control, fast retransmit, and recovery.

1.6 VERSION-WISE REFERENCE BACKBONE

Each version family is grounded in a small set of reference sources that shaped its design and implementation style.

Each version family is based on a small number of reference sources that influenced its design and implementation style. This phased model offers two main benefits. First, debugging stays manageable since each level brings in a small set of new behaviors. Second, the connection between protocol features and observed performance becomes clear in experiments.

Version Family	**Main Ideas**	**Reference Backbone and Code Inspiration**
v0–v2	UDP bit/byte transfer, reconstruction, end-of-message handling	RFC 768 for UDP, Beej's Guide for socket programming basics, Stevens for implementation discipline, Kurose and Ross for protocol layering Postel (1980); Hall (2024); Stevens and Wright (1995); Kurose and Ross (2021)
v3–v7	Parity, sequence awareness, ACK/NAK, stop-and-wait reliability	Kurose and Ross for reliable transport concepts, Tanenbaum and Wetherall for error detection and protocol reasoning, Stevens for practical C socket implementation Kurose and Ross (2021); Tanenbaum and Wetherall (2011); Stevens and Wright (1995)
v8–v14	Handshake, checksum, sliding window, selective-repeat style buffering, adaptive timeout, fast retransmit	RFC 793, RFC 6298, Kurose and Ross, and Stevens for transport semantics and timer design Postel (1981); Paxson *et al.* (2011); Kurose and Ross (2021); Stevens and Wright (1995)
v15–v24	Adaptive control, RTT tuning, timestamps, SACK-like ideas, congestion avoidance, educational Reno-style recovery	RFC 5681, RFC 2018, RFC 7323, RFC 6298, RFC 793, Stevens, and Kurose and Ross Allman *et al.* (2009); Mathis *et al.* (1996); Borman *et al.* (2014); Paxson *et al.* (2011); Postel (1981); Stevens and Wright (1995); Kurose and Ross (2021)

Table 1.1: Main references used for each protocol family.

1.7 EDUCATIONAL VALUE

From an academic viewpoint, this project offers a practical link between computer networks, systems programming, and algorithm design. Key ideas from textbooks become tangible:

1. **Data integrity:** parity and checksum validation.
2. **Reliability:** acknowledgment and retransmission.
3. **Ordering:** sequence numbers and expected sequence tracking.
4. **Pipelining:** multiple in-flight packets through windows.
5. **Flow under uncertainty:** adaptive timeout and congestion response.

Since each feature is implemented manually, learners can see trade-offs directly. For instance, stop-and-wait is simple to understand, but it is inefficient at higher latency. Sliding windows boost throughput but increase state complexity. Selective-repeat buffering enhances reliability under reordering but requires careful tracking.

The interactive visualizer further adds educational value by displaying the sender code, receiver code, finite state machine, and UML timeline together. This approach makes it easier for beginners to grasp protocol state transitions as they learn transport concepts for the first time.

1.8 SCOPE AND BOUNDARIES

The project is designed for learning and controlled experiments. Important boundaries are listed below.

1. Communication is mainly localhost based.
2. Address and port are hardcoded in most versions.
3. Security features like encryption and authentication are not included.
4. Receiver-advertised window and full SACK format are not fully implemented.
5. Most versions handle one sender-receiver flow at a time.

These boundaries are acceptable for a graduation project focused on protocol mechanics. They also provide clear directions for future extension.

1.9 RESEARCH QUESTIONS

The implementation and analysis are organized around practical questions:

1. How much reliability can be achieved using only user-space logic over UDP?
2. At which stages does throughput improve significantly, and why?
3. How does adaptive timeout outperform fixed timeout under variable delays?
4. How does congestion window control affect retransmission behavior and completion time?
5. How can students use AI tools responsibly for protocol learning (idea-level guidance and debugging support) while preserving original implementation ownership and textbook-grounded correctness?

CHAPTER 2

PROTOCOL DESIGN AND IMPLEMENTATION

In this chapter, we will discuss the complete version to version protocol evolution. The Strategy we will implement is incremental. Each version of the protocol retains all of the capabilities from prior versions, however each new version introduces at least one carefully selected feature. Thus, this method provides increased source correctness and also provides a greater practical capability to debug individual implementations.

2.1 GLOBAL DESIGN PRINCIPLES

The same design principles are followed across all versions.

1. **One focus at a time:** each version teaches one protocol idea.
2. **Clear state:** sender and receiver keep simple names for sequence numbers, timeouts, and window state.
3. **Predictable moves:** each packet leads to a known state change.
4. **Easy to trace:** console output makes each test run easier to follow.

2.2 VERSION FAMILY A: BIT AND BYTE TRANSFER (V0–V2)

The first family introduces communication mechanics at the lowest useful level.

2.2.1 Version 0: Single-Bit UDP Demo

In level 0, the sender sends one bit to the receiver using a UDP datagram. There is no framing beyond the single value. This version validates:

1. create the socket,
2. point it to the receiver,
3. send the datagram,
4. receive the datagram.

This step is intentionally minimal and prepares the path for structured packet design.

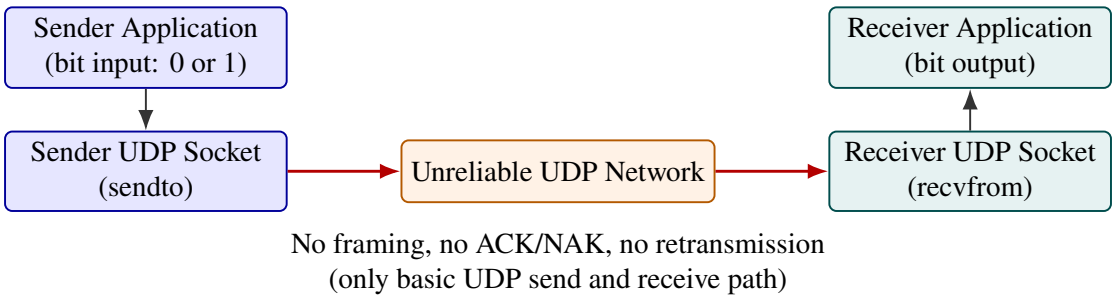

Figure 2.1: Version 0: one bit sent over UDP.

2.2.2 Version 1: Character as 8 Independent Bits

In level 1, one character is encoded into 8 bits and sent as 8 UDP datagrams. The receiver reconstructs the byte by shifting and OR operations. Timing is regulated using small sleep intervals so packet order remains easier to track in local runs.

Educational value:

1. seeing how binary values are stored,
2. learning why bit order matters,
3. rebuilding the byte at the receiver.

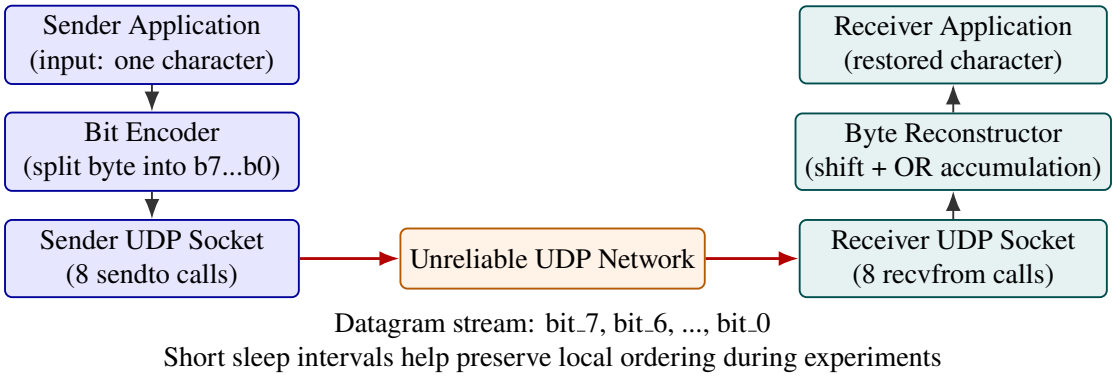

Figure 2.2: Version 1 architecture: one character transmitted as eight UDP bit datagrams and reconstructed at receiver side.

2.2.3 Version 2: Full String by Bit Stream

Level 2 extends the bit stream approach to full strings. A null character is sent at the end to flag message completion. At this stage, communication works for complete user input, but no error detection or reliability control exists.

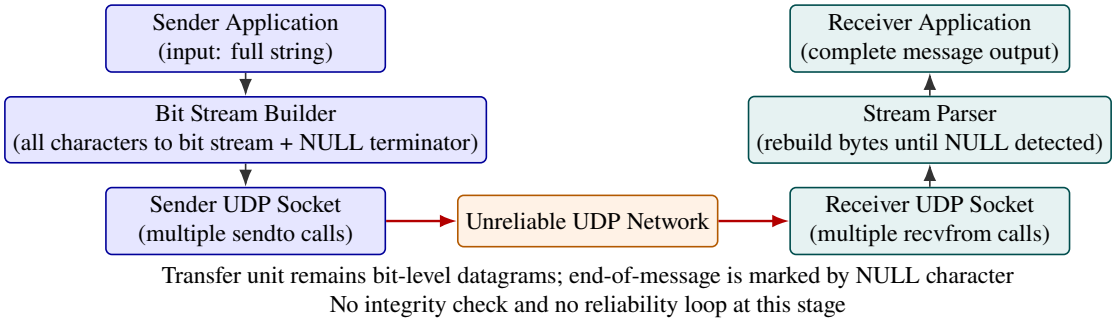

Figure 2.3: Version 2 architecture: full string transmitted as a bit stream with NULL-terminated completion.

2.3 VERSION FAMILY B: ERROR DETECTION AND STOP-AND-WAIT (V3–V7)

Family B introduces reliability fundamentals.

2.3.1 Version 3: Parity Detection

For each byte, sender computes parity and transmits it with payload bits. Receiver recomputes parity and compares values.

Outcome:

1. corruption can be detected,
2. corruption cannot yet be corrected.

Parity is simple and fast but weak against many multi bit error patterns.

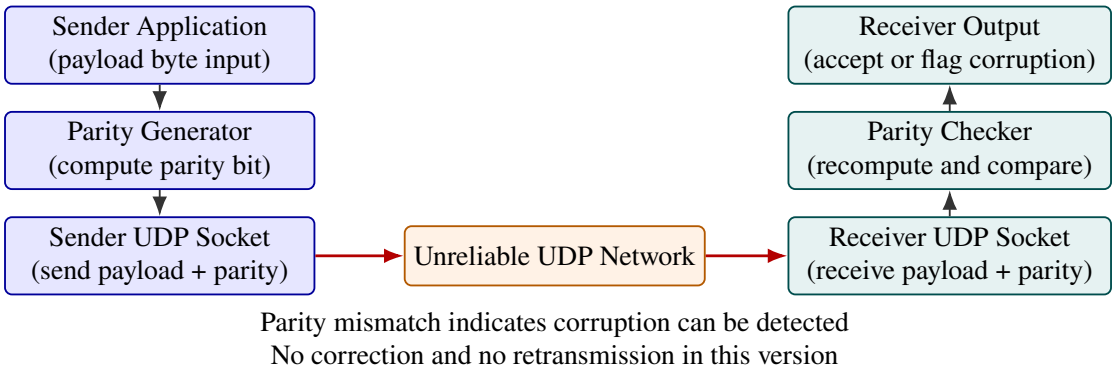

Figure 2.4: Version 3 architecture: parity based corruption detection over UDP payload transfer.

2.3.2 Version 4: Sequence Numbers

Level 4 attaches sequence numbers to per character transfer. This improves observability and creates the basis for reliable acknowledgement logic.

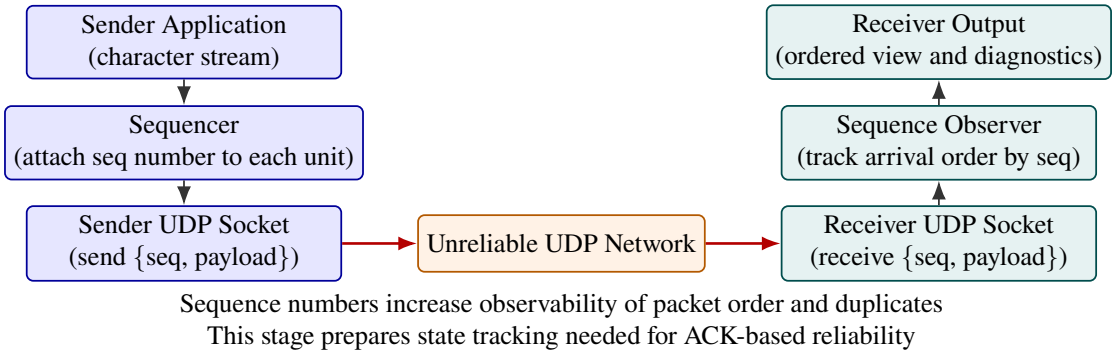

Figure 2.5: Version 4 architecture: sequence numbered UDP units for order visibility and reliability groundwork.

2.3.3 Version 5: Stop-and-Wait ARQ with ACK/NAK

In level 5, receiver sends ACK on success and NAK on parity failure. Sender transmits one unit and waits for response before moving forward. This is the first true reliability loop.

Strength:

1. simple correctness argument,
2. easy handling of duplicates,
3. straightforward debugging.

Limitation:

1. low throughput because channel stays idle while waiting.

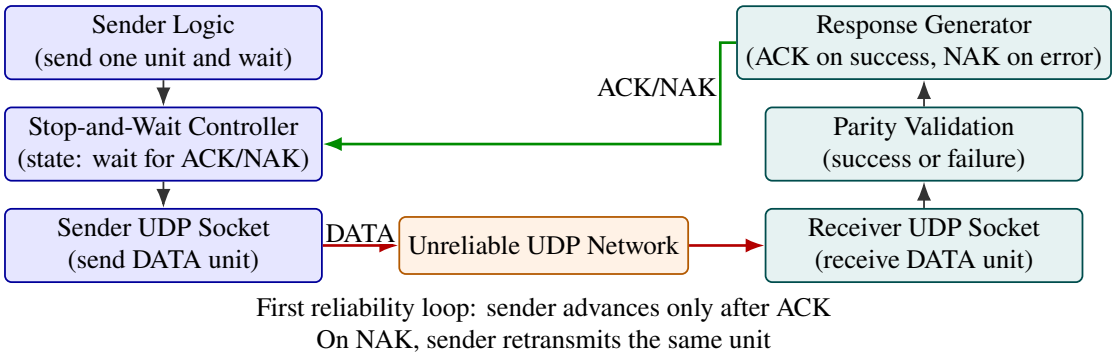

Figure 2.6: Version 5 architecture: stop-and-wait ARQ with ACK/NAK feedback path.

2.3.4 Version 6: Timeout and Retries

Level 6 adds socket receive timeout and retry limit. If ACK does not arrive in time, sender retransmits. Receiver tracks expected sequence and handles duplicates conservatively.

This version is the first practically robust design under packet loss.

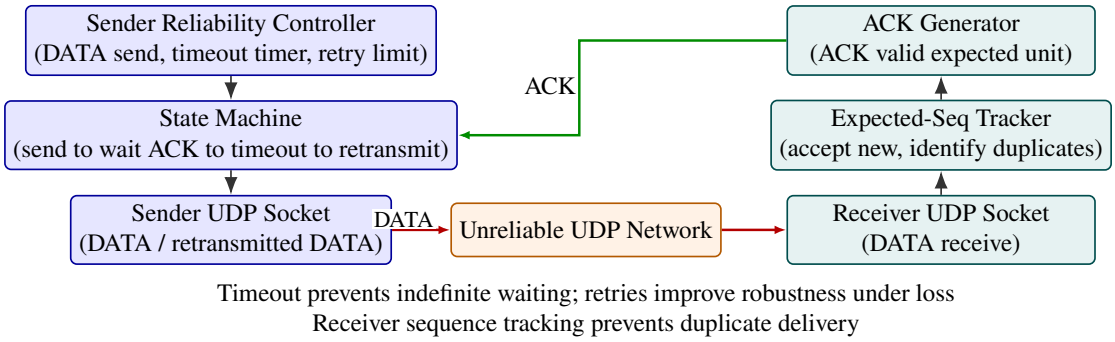

Figure 2.7: Version 6 architecture: timeout driven retransmission with duplicate safe receiver handling.

2.3.5 Version 7: Struct Based Packetization

Version 7 replaces loose fields with an explicit packet structure:

1. sequence number,
2. payload,
3. parity,
4. end flag.

This change improves code readability and prepares the architecture for multi-field TCP-like packets in later levels.

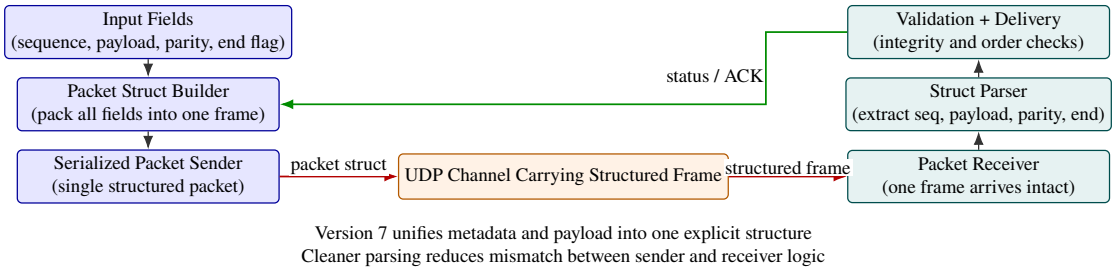

Figure 2.8: Version 7 architecture: struct based packetization with unified sender and receiver parsing flow.

2.4 VERSION FAMILY C: TCP-LIKE FRAMING AND PIPELINING (V8–V10)

This family introduces transport semantics closer to TCP.

2.4.1 Version 8: Handshake, Flags, and Connection Lifecycle

Level 8 includes:

1. SYN for start,
2. ACK for acknowledgement,
3. DATA for payload,
4. FIN for graceful close.

The sender and receiver perform a simplified three way handshake and teardown. Checksum is also introduced at this level for stronger integrity checks.

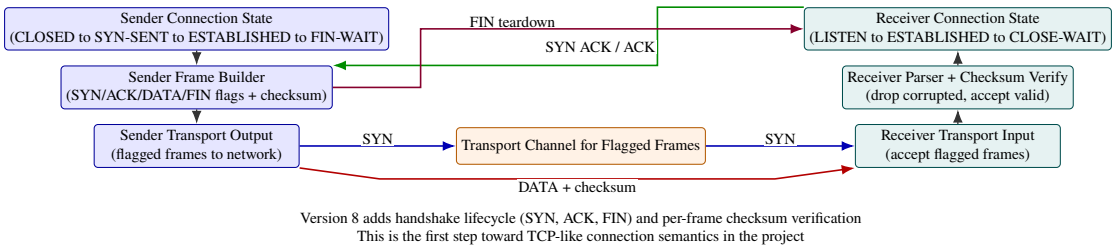

Figure 2.9: Version 8 architecture: handshake flags, lifecycle control, and checksum protected data flow.

2.4.2 Version 9: Sliding Window (Go-Back-N Style)

In version 9, an increase in efficiency has been implemented using a ”sender window” (4 packets typically), where the sender can send multiple packets before receiving an ACK. If the sender does not receive an ACK after a timeout, it will re-send packets from the base. The receiver will also use a cumulative ACK-style mechanism to acknowledge the last-sequence number received by the sender.

This level marks a major throughput jump because waiting time is amortized.

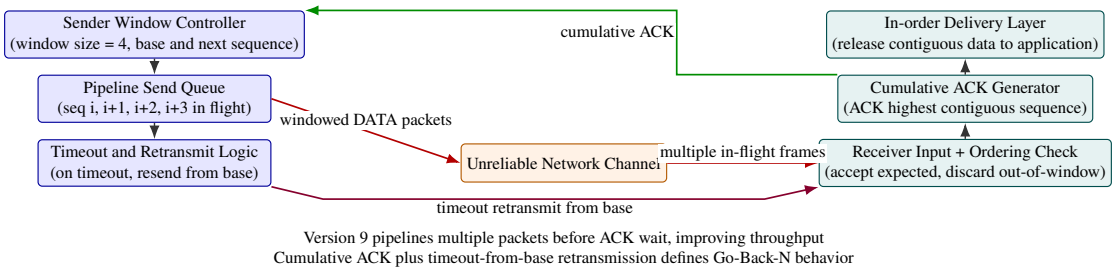

Figure 2.10: Version 9 architecture: sliding window pipelining with cumulative ACK and timeout based retransmission.

2.4.3 Version 10: Chunk-Based Payloads

Version 10 increases data per packet by introducing `CHUNK_SIZE = 8` bytes. Benefits:

1. fewer packets for same message length,
2. less per packet header overhead,
3. better throughput for moderate messages.

Receiver logic starts supporting in window buffering, enabling transition to selective-repeat style delivery in the next family.

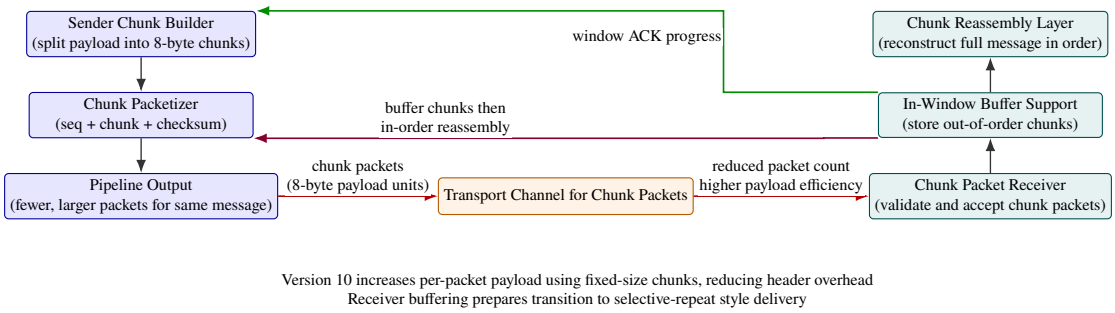

Figure 2.11: Version 10 architecture: chunk based payload transfer with in-window buffering and ordered reassembly.

2.5 VERSION FAMILY D: SELECTIVE REPEAT AND ADAPTIVE CONTROL (V11–V14)

Family D contains the most advanced protocol behavior in this project.

2.5.1 Version 11: Selective-Repeat Like Buffering

Receiver stores out-of-order chunks in buffers and delivers data in order when missing sequence arrives. This avoids unnecessary retransmission of received data and improves behavior under reordering.

Key receiver structures include:

1. packet received flags,
2. chunk data buffer,
3. chunk length metadata,
4. expected sequence tracker.

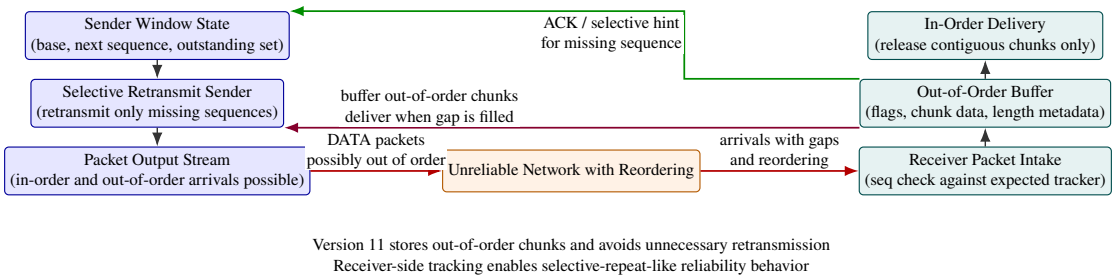

Figure 2.12: Version 11 architecture: selective-repeat like buffering with out-of-order storage and in-order delivery.

2.5.2 Version 12: Adaptive RTO Estimation

Version 12 introduces RTT sampling and adaptive RTO calculation. The fixed timeout mechanism does not perform well when the delay between sending and receiving packets is variable. The adaptive timeout mechanism adapts the retransmit threshold dynamically, thereby reducing both premature retransmissions and prolonged idle periods before initiating a retransmission.

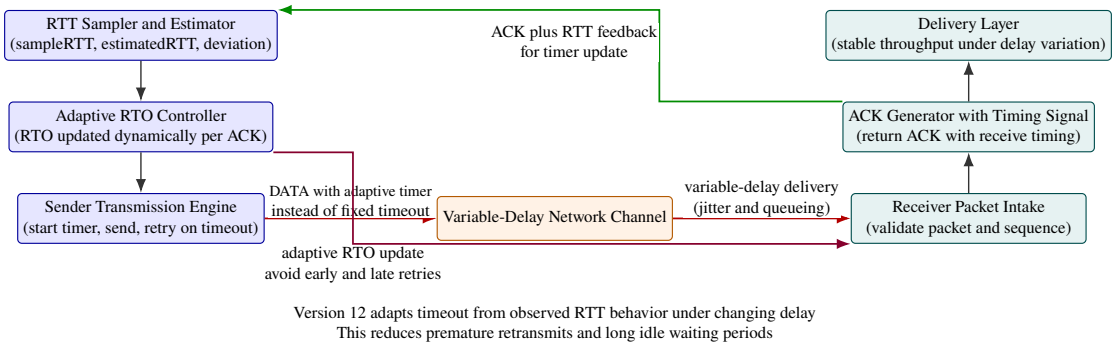

Figure 2.13: Version 12 architecture: RTT sampling with adaptive RTO control for variable delay conditions.

2.5.3 Version 13: Congestion Window and Slow Start

This level introduces congestion control variables:

1. `cwnd` (congestion window),
2. `ssthresh` (slow start threshold).

Behavior follows a simplified policy:

1. grow quickly in slow start,
2. grow gradually in congestion avoidance,
3. reduce sending rate after loss indication.

This improves fairness and stability compared to aggressive fixed-window sending.

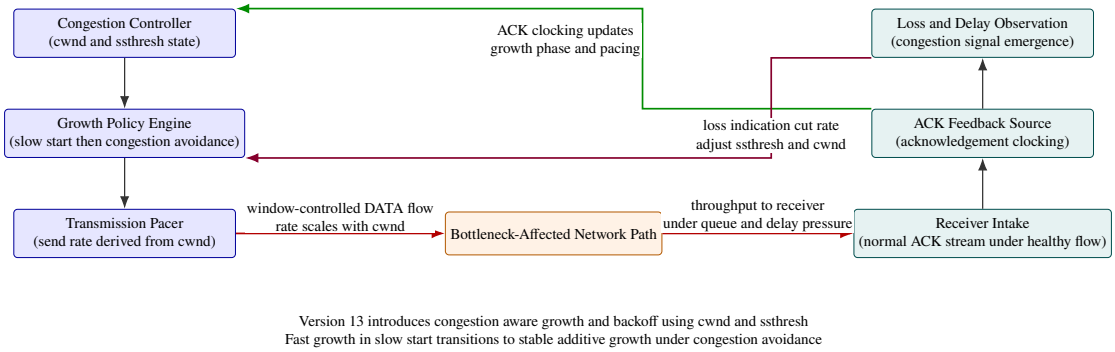

Figure 2.14: Version 13 architecture: congestion window growth, ACK clocking, and loss triggered rate reduction.

2.5.4 Version 14: Fast Retransmit and Fast Recovery

Version 14 is the most complete implementation within Family D. The project then continues through versions 15–24, which extend this baseline with more advanced adaptive and Reno like refinements. Enhancements include:

1. duplicate ACK counting,
2. triple duplicate ACK loss inference,
3. fast retransmission without waiting for timeout,
4. temporary fast recovery window inflation and controlled deflation.

This behavior closely resembles a learning scale version of TCP Reno.

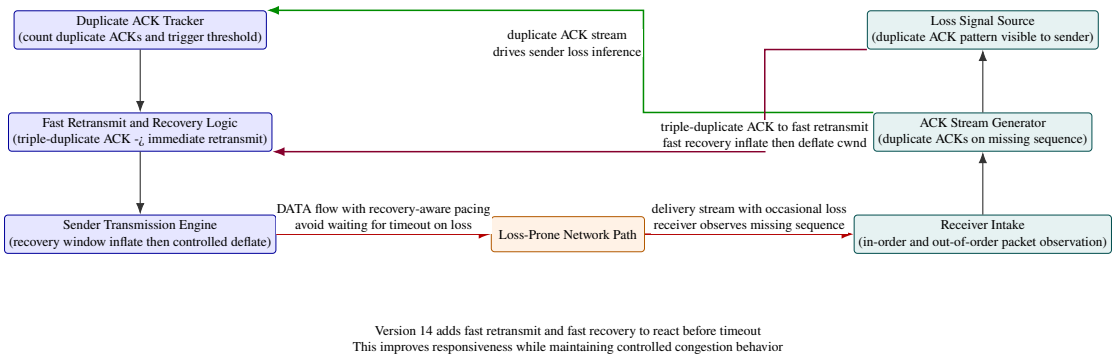

Figure 2.15: Version 14 architecture: duplicate ACK loss inference with fast retransmit and fast recovery control.

2.6 VERSION FAMILY E: ADAPTIVE AND RENO LIKE EXTENSIONS (V15–V24)

The latter versions advance the transport process into a more sophisticated and educational TCP-like environment. This family emphasizes RTT-sensitive refinement of timeouts, state awareness based on timestamps, SACK-like reasoning, avoiding congestion, and improved recovery procedure. The visualizer uses later versions of the project to demonstrate on a step by step basis how the FSM and UML timeline can assist in explaining protocol decisions.

The RFCs and texts that describe TCP in practice provide the main design inspiration for this family. The RFC 6298 describes the retransmission timer, RFC 5681 describes congestion avoidance, RFC 2018 provides options for selective acknowledgment (SACK). TCP extensions for high speed are found in RFC 7323, and RFC 793 provides the core TCP model. Additionally, the Stevens, Kurose and Ross, and Tanenbaum and Wetherall texts (used for the implementation style) make it easier to relate TCP theory to the actual sender and receiver code structure used in the project Passport et al. (2011); Allman et al. (2009); Mathis et al. (1996); Borman et al. (2014); Postel (1981); Stevens and Wright (1995); Kurose and Ross (2021); Tanenbaum and Wetherall (2011).

This family is very useful to the report because it demonstrates the moment when the learning project approaches real TCP thought processes both visually and conceptually.

2.6.1 Version 15: RTT-Smoothed Timeout Refinement

Version 15 stabilizes retransmission timing using smoothed RTT and deviation tracking. Instead of relying on coarse timeout behavior, this version updates RTO with a more disciplined estimator so sender retries are less noisy.

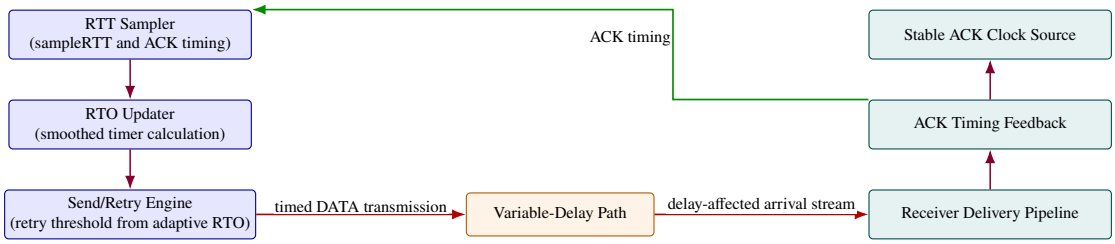

Figure 2.16: Version 15 architecture: smoothed RTT driven retransmission timeout refinement.

2.6.2 Version 16: Adaptive Window Discipline

Version 16 connects timeout behavior with sender pacing and window discipline. When timing uncertainty grows, send aggressiveness is reduced. When ACK rhythm stabilizes, throughput is allowed to grow safely.

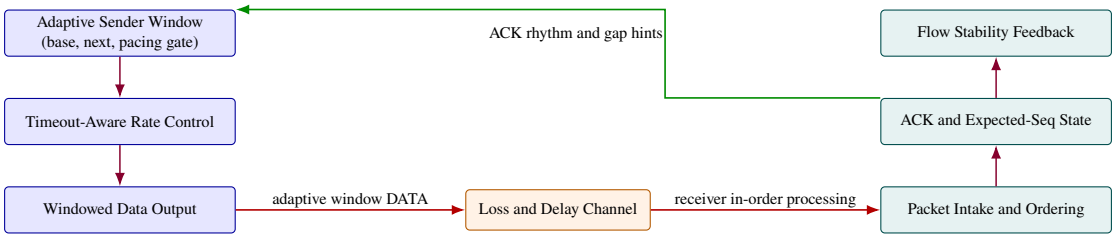

Figure 2.17: Version 16 architecture: timeout aware adaptive window control.

2.6.3 Version 17: Timestamp Option Integration

Version 17 introduces timestamp style metadata in packet exchange so sender can infer timing more directly and avoid ambiguous retransmission decisions.

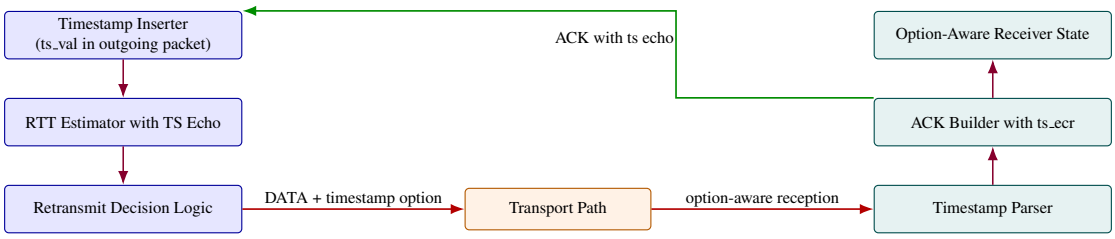

Figure 2.18: Version 17 architecture: timestamp insertion and echo aware feedback loop.

2.6.4 Version 18: SACK-Inspired Receiver Reporting

Version 18 introduces selective acknowledgment style hints so sender can see which blocks arrived beyond a gap. This reduces unnecessary retransmission of already received data.

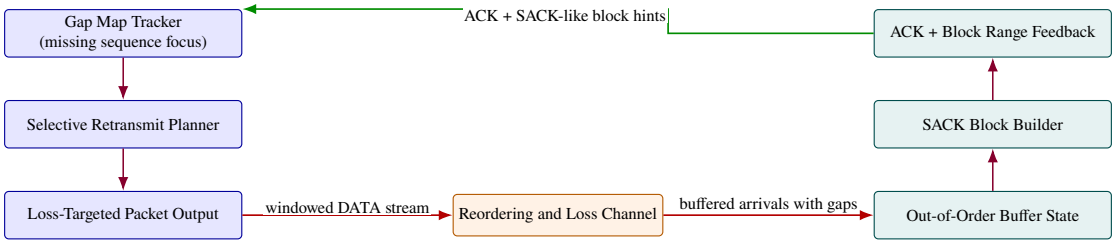

Figure 2.19: Version 18 architecture: SACK-inspired gap reporting and selective resend planning.

2.6.5 Version 19: Congestion Avoidance Stabilization

Version 19 strengthens congestion avoidance transitions. The sender responds to loss and delay pressure with smoother cwnd behavior, reducing oscillation while keeping reasonable throughput.

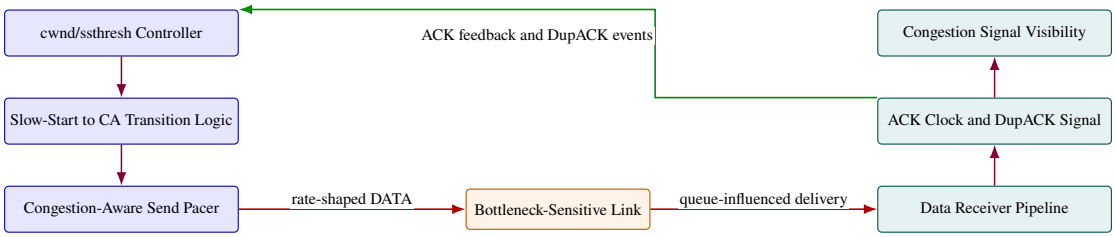

Figure 2.20: Version 19 architecture: stabilized congestion avoidance behavior with ACK clocked feedback.

2.6.6 Version 20: Loss-Recovery Coupled with Option Feedback

Version 20 combines fast recovery behavior with richer feedback from timestamp and selective-ack style information. Recovery targets missing data more precisely and returns to normal transmission with better control.

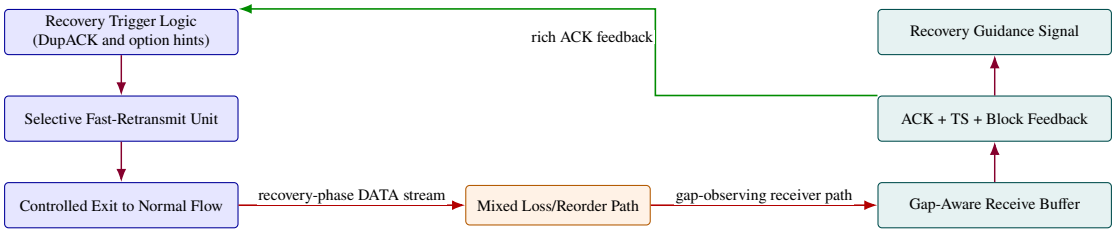

Figure 2.21: Version 20 architecture: fast recovery guided by richer feedback signals.

2.6.7 Version 21: Sender State Consolidation

Version 21 consolidates sender logic for timers, congestion state, option tracking, and retransmission queues so control decisions remain consistent across difficult traffic patterns.

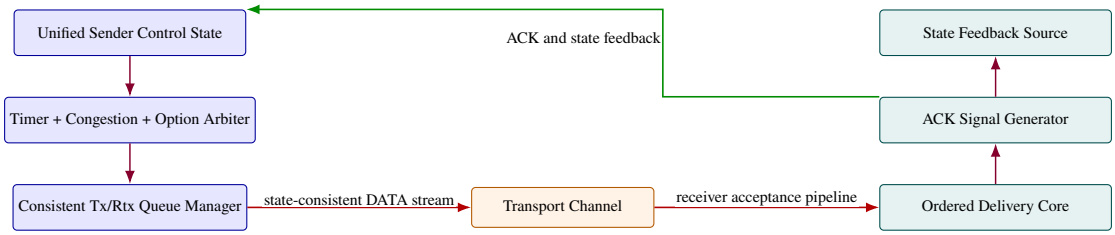

Figure 2.22: Version 21 architecture: consolidated sender control for stable decision flow.

2.6.8 Version 22: Receiver Reassembly and Delivery Hardening

Version 22 strengthens the receiver side by improving buffering, gap filling, and reassembly commit rules. This reduces edge case delivery mistakes when packets arrive late, duplicated, or out of order.

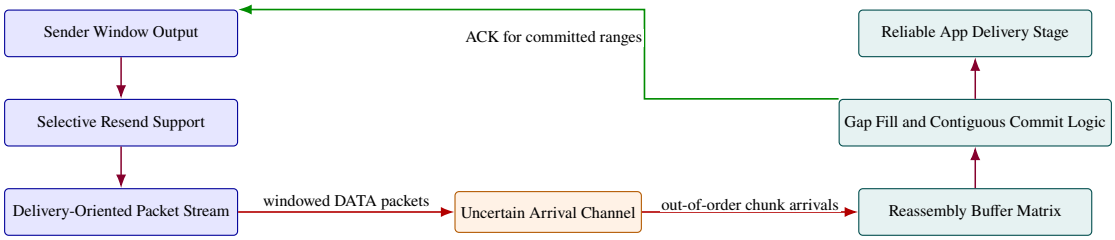

Figure 2.23: Version 22 architecture: hardened receiver reassembly and contiguous delivery control.

2.6.9 Version 23: End-to-End Robustness Tuning

Version 23 tunes interaction among timer control, congestion response, selective retransmission, and receiver commit rules. The focus is robust behavior under mixed delay, reordering, and burst loss.

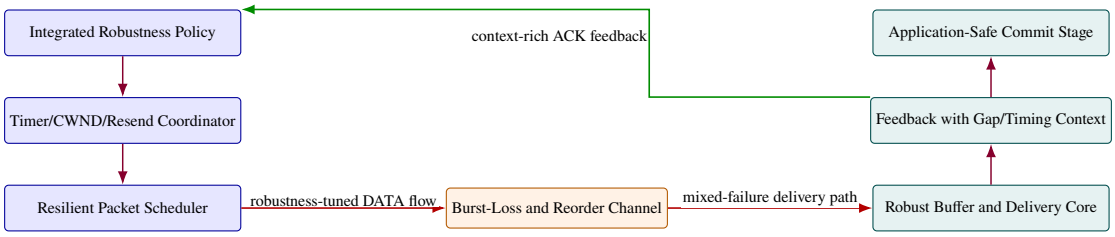

Figure 2.24: Version 23 architecture: coordinated end-to-end robustness tuning under mixed failures.

2.6.10 Version 24: Final Integrated Transport Stage

Version 24 is the final integrated stage of the project. It combines the full set of educational transport features: adaptive timing, congestion aware windows, timestamp aware feedback, SACK inspired loss hints, and controlled recovery behavior.

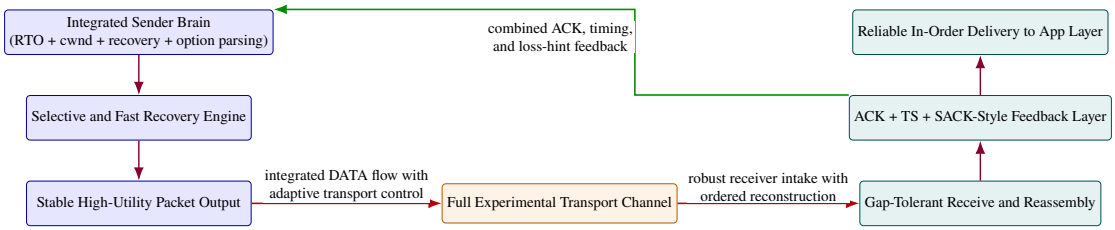

Figure 2.25: Version 24 architecture: final integrated transport design across reliability, timing, and recovery.

2.6.11 Family E Summary

Versions 15–24 complete the transition from a staged classroom transport model to an integrated TCP-inspired system. This illustrates how the use of timing, selective feedback, congestion awareness and systematic recovery can be added in layers while maintaining traceability for learners.

2.7 CHAPTER SUMMARY

This chapter described the stage changes of the entire project starting from merely a UDP demo until eventually evolving into a comprehensive transport protocol lab. The following chapter details the mathematical algorithms and equations used to define the process of implementation from a quantitative perspective. Finally, Chapter 4 will conclude with all of the empirical data, analysis and visual timeline findings to finalize the educational narrative.

CHAPTER 3

ALGORITHMIC AND QUANTITATIVE ANALYSIS

This chapter explains the main algorithms used in the project and what they do in practice. The formulas are here to show how the code works, not just to list math on its own.

3.1 BIT ENCODING AND MESSAGE COST

In early versions, each character is broken into 8 bits and transmitted as separate UDP datagrams. If a message has length L characters, then the number of payload datagrams in pure bit mode is

$$N_{\text{bit}} = 8L + 8, \tag{3.1}$$

where the additional 8 corresponds to transmitting the null terminator as one more byte.

That is why bit mode is slow. The number of datagrams grows very quickly.

3.2 PARITY-BASED ERROR DETECTION

For versions with parity, sender computes parity of one byte:

$$p = b_0 \oplus b_1 \oplus \cdots \oplus b_7, \tag{3.2}$$

where b_i is the ith bit of the character and $\oplus$ denotes XOR.

The receiver checks parity again and compares the result. If there is a mismatch, that packet is marked as corrupted. Parity catches all odd bit errors but not every even one, which is why later versions move to checksum.

3.3 STOP-AND-WAIT SERVICE TIME

In stop-and-wait, sender transmits one unit and waits for ACK before next send. Approximate per packet cycle time is

$$T_{\text{cycle}} = T_{\text{tx}} + T_{\text{prop}} + T_{\text{proc}} + T_{\text{ack}}. \tag{3.3}$$

Ideal throughput under stop-and-wait is therefore bounded by

$$\text{Throughput}_{\text{SAW}} \approx \frac{\text{payload per packet}}{T_{\text{cycle}}}. \tag{3.4}$$

Even when channel capacity is high, waiting for each ACK keeps throughput low if the round trip delay is not small.

3.4 CHECKSUM IN TCP-LIKE VERSIONS

In advanced versions, a one's complement style checksum is computed over key header fields and payload bytes. In simple terms:

$$S = \sum (\text{header fields} + \text{payload bytes}), \tag{3.5}$$

$$S' = (S \bmod 2^{16}) + \left\lfloor \frac{S}{2^{16}} \right\rfloor, \tag{3.6}$$

$$\text{checksum} = \sim S'. \tag{3.7}$$

At the receiver, checksum field is reset, recomputed, and compared. This gives stronger detection than parity and supports multi byte payload packets.

3.5 SLIDING WINDOW EFFICIENCY

With sender window size W, up to W packets can be in flight. In an ideal pipeline, normalized channel utilization can be approximated as

$$U \approx \min\left(1, \frac{W}{1+2a}\right), \tag{3.8}$$

where $a = T_{\text{prop}}/T_{\text{tx}}$.

This is why a larger window increases throughput until the path is fully used.

3.6 CHUNKING GAIN

If payload per packet increases from 1 byte to C bytes (for example, $C = 8$), the number of data packets for message length L becomes

$$N_{\text{chunk}} = \left\lceil \frac{L}{C} \right\rceil. \tag{3.9}$$

Compared with one-byte payloads, the packet count drops by about

$$\text{Gain factor} \approx C, \tag{3.10}$$

ignoring control packets and retransmissions.

3.7 ADAPTIVE RTO ESTIMATION

Versions 12 and above use smoothed RTT estimation and variation based timeout. Given sample RTT R, smoothed RTT ($SRTT$) and RTT variation ($RTTVAR$) are updated as follows:

$$RTTVAR \leftarrow (1-\beta)\, RTTVAR + \beta\, |SRTT - R|, \tag{3.11}$$

$$SRTT \leftarrow (1-\alpha)\, SRTT + \alpha\, R. \tag{3.12}$$

Then retransmission timeout is estimated by

$$RTO = SRTT + K \cdot RTTVAR, \tag{3.13}$$

and bounded in implementation between predefined minimum and maximum values.

This model is aligned with practical transport engineering guidance and avoids both aggressive and overly delayed retransmissions.

3.8 CONGESTION WINDOW DYNAMICS

In congestion controlled versions, sender maintains congestion window $cwnd$ and slow-start threshold $ssthresh$.

3.8.1 Slow Start

When $cwnd < ssthresh$, growth is fast (approximately one MSS equivalent unit per ACK, resulting in near exponential growth per RTT in simplified models).

3.8.2 Congestion Avoidance

When $cwnd \geq ssthresh$, growth becomes gradual, approximating additive increase.

3.8.3 Loss Response

On timeout event:

$$ssthresh \leftarrow \max\left(2, \frac{cwnd}{2}\right), \tag{3.14}$$

$$cwnd \leftarrow 1. \tag{3.15}$$

On triple duplicate ACK in fast retransmit logic:

$$ssthresh \leftarrow \max\left(2, \frac{cwnd}{2}\right), \tag{3.16}$$

$$cwnd \leftarrow ssthresh + 3. \tag{3.17}$$

After successful recovery, window is deflated near threshold level. This avoids fully restarting from 1 in many recoverable loss cases.

3.9 LATENCY COMPONENTS

Observed transfer completion time includes multiple components:

$$T_{\text{total}} = T_{\text{setup}} + T_{\text{data}} + T_{\text{retrans}} + T_{\text{teardown}}. \tag{3.18}$$

Where:

1. T_{setup} includes handshake steps,
2. T_{data} includes normal in order transfer,
3. T_{retrans} includes recovery delay from loss/corruption,
4. T_{teardown} includes connection close signaling.

This decomposition helps explain why different versions show different completion times for the same message length.

3.10 RELIABILITY METRICS

Useful metrics for experiment reporting are defined below.

$$\text{Delivery Success Rate} = \frac{\text{Successfully delivered messages}}{\text{Messages attempted}}, \tag{3.19}$$

$$\text{Retransmission Ratio} = \frac{\text{Retransmitted packets}}{\text{Original packets}}, \tag{3.20}$$

$$\text{Goodput} = \frac{\text{Useful payload bits delivered}}{T_{\text{total}}}. \tag{3.21}$$

These metrics give a balanced view across correctness and efficiency.

3.11 A PRACTICAL THEOREM-LIKE OBSERVATION

Theorem 3.1. *Increasing the size of sender's window increases throughput up to the saturation point and, thereafter, throughput is limited.*

Proof. Increasing the number of packets in transmission reduces total time sender waits on acknowledgement before sending the next packet in sequence. The larger the packet window, the more packets can be transmitted before sender has to wait on an acknowledgement; and, therefore, the fewer times the sender will be idle. However, once the number of in transit packets is equal to the product of the time it takes for a packet to travel from sender to receiver and the receiver's ability to process packets (e.g. memory); no increase in sender's window will produce any additional packets delivered. Thus, when there are lost packets in error, a larger sending window will produce a larger error recovery burden unless recipient uses selective acknowledgment. Consequently, throughput will plateau once sender reaches maximum size of window. ■

3.12 INTERPRETATION FOR THIS PROJECT

The formulas in this chapter explain observed behavior from level progression:

1. versions v0–v2 have very high packet overhead,
2. v5–v7 improve correctness but remain latency bound,
3. v9 and above improve utilization through pipelining,
4. v10 and above reduce packet count through chunking,
5. v12 and above improve timing decisions through adaptive RTO,
6. v13 and above balance aggressiveness and stability using congestion control logic and recovery aware retransmission behavior,
7. v15–v24 extend the same transport ideas with richer TCP-like refinements, including timestamp and recovery oriented control.

This quantitative interpretation supports the implementation narrative in the previous chapter and sets context for the comparative results in the next chapter.

CHAPTER 4

ILLUSTRATIONS, COMPARATIVE ANALYSIS, AND FINAL OUTCOMES

This chapter shows the visuals and final takeaways from version 0 to version 24.

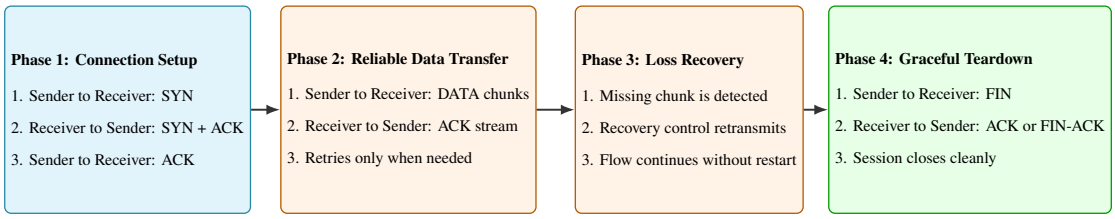

Figure 4.1: Full session flow for the advanced versions.

4.1 END-TO-END PROTOCOL LIFECYCLE

Figure 4.1 shows the main steps: setup, data transfer, acknowledgement, and clean teardown. That flow is the basis for comparing versions.

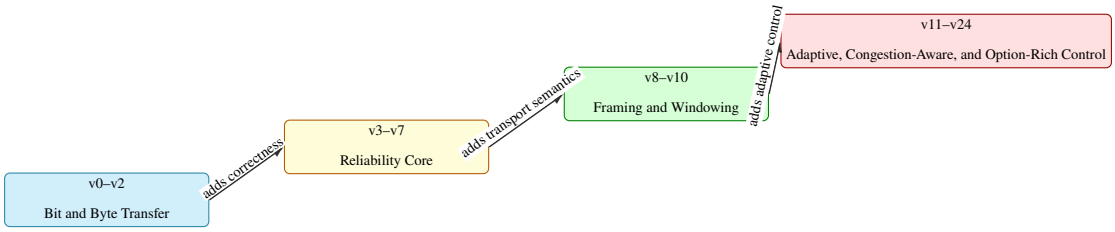

Figure 4.2: How the feature set grows across version families.

4.2 VERSION CAPABILITY LAYERS

Figure 4.2 shows how each family adds one clear group of transport features at a time.

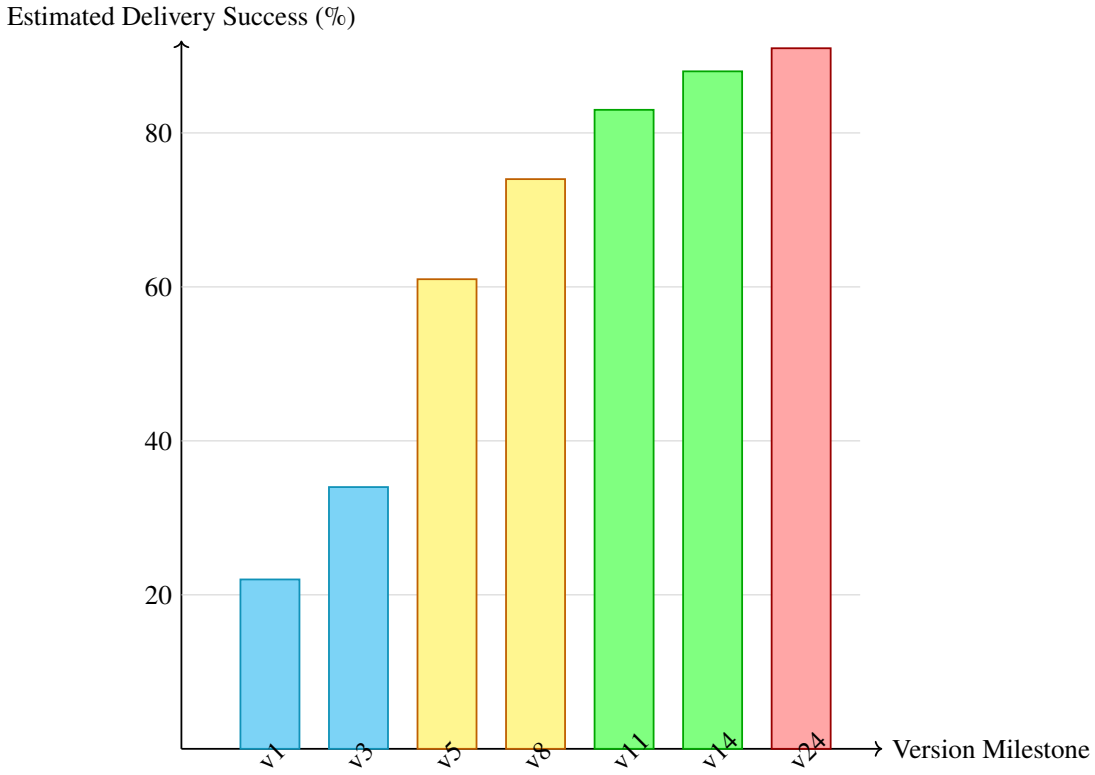

Figure 4.3: Reliability trend as the protocol grows.

4.3 RELIABILITY PROGRESS CHART

Figure 4.3 shows how successful delivery improves as checks, acknowledgements, retries, buffering, and adaptive control are added.

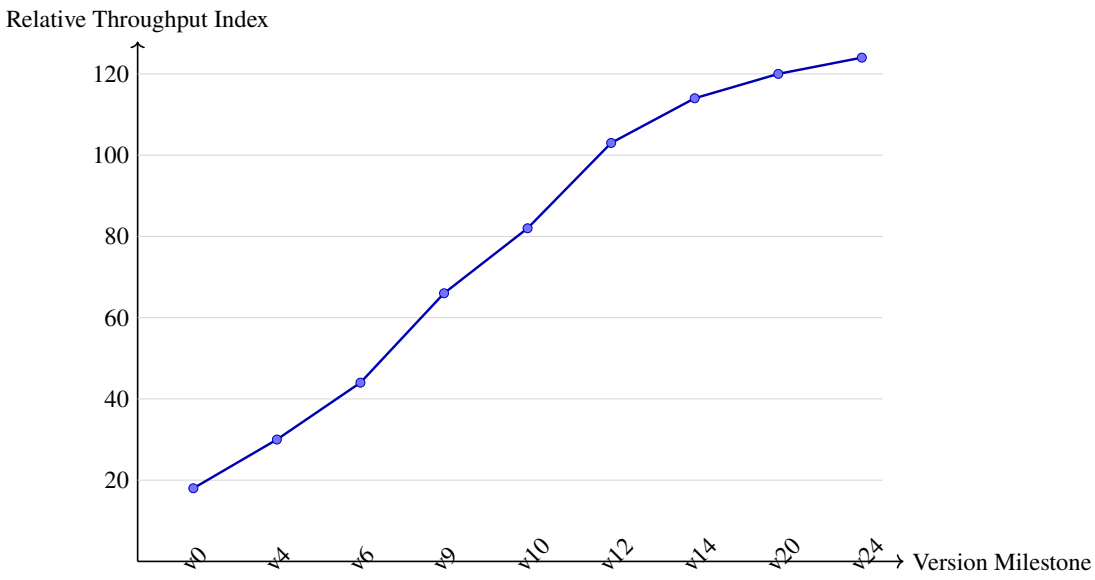

Figure 4.4: Throughput trend across the staged versions.

4.4 THROUGHPUT-ORIENTED IMPROVEMENT CHART

Figure 4.4 shows why pipelining, chunking, and faster loss recovery improve channel use.

Table 4.1: Feature comparison across key protocol milestones.

Capability	**v2**	**v5**	**v8**	**v11**	**v14**	**v24**
Bit/byte transfer	Yes	Yes	Yes	Yes	Yes	Yes
Parity/Checksum integrity	No	Parity	Checksum	Checksum	Checksum	Checksum + options
ACK-based reliability	No	Stop-and-wait	Yes	Yes	Yes	Yes + SACK-like hints
Sliding window send	No	No	Yes	Yes	Yes	Yes (adaptive)
Out-of-order buffering	No	No	No	Yes	Yes	Yes
Adaptive retransmission timer	No	No	No	Yes	Yes	Yes (RTT tuned)
Congestion control response	No	No	No	Partial	Yes	Stronger/consistent
Fast retransmit / recovery	No	No	No	No	Yes	Yes (refined with later logic)
Timestamp / option awareness	No	No	No	No	No	Yes

4.5 COMPARATIVE FEATURE MATRIX

Table 4.1 condenses the technical progression into a single reference view for discussion and evaluation.

Figure 4.5: Fast loss recovery in the advanced versions.

4.6 LOSS-RECOVERY BEHAVIOR ILLUSTRATION

Figure 4.5 shows the main advantage of later versions: duplicate ACKs can trigger recovery before the timeout runs out.

4.7 KEY FINDINGS

Five clear takeaways come from the visuals and tables:

1. Acknowledgements and timeouts make the system much more reliable.
2. Chunked sending and sliding windows are what push throughput up.
3. Receiver buffering in v11+ reduces waste when packets arrive out of order.
4. Adaptive RTO cuts down unnecessary retransmissions when delay changes.
5. Fast retransmit/recovery, first added in v14 and refined through v24, reacts to loss faster than timeout-only recovery.

4.8 LIMITATIONS

1. Most tests ran on localhost, so they do not show full WAN variety.
2. Security features such as authentication and encryption are left out on purpose.
3. A full TCP-style advertised window and SACK block encoding are out of scope here.
4. The charts are useful for analysis, but a fully automated benchmark harness is still future work.

4.9 CONCLUSION

This chapter shows that the project goes from simple datagram exchange to a strong educational transport model that is reliable, can handle multiple requests at once, can adapt, and is aware of congestion. The final versions are useful for both putting into practice and learning about transport protocols.

One of the most important final results is the interactive visualizer. It brings together the sender code, receiver code, finite state machine, and UML timeline into one clear educational view. That makes the protocol easier for new students to learn, since TCP, congestion control, and C-based socket programming can be hard to understand when you only read about them or look at code. In this way, the project is more than just a piece of software; it's also a useful teaching tool that makes the ideas clear, organized, and easier to remember.

As the last step in the project, I made both the protocol code and the interactive visualizer for use in the classroom. Students find it easier to understand networking concepts when they have versioned C implementations, textbook-based references, and synchronized visual explanations. This is especially helpful for beginners because the visual walkthrough makes C and transport-protocol ideas easier to understand and turns the system into a step-by-step learning experience.

APPENDIX A

EXTENDED TECHNICAL APPENDIX

This appendix provides additional implementation depth for the TCP/IP-inspired UDP project. It is written to support evaluation, viva discussion, and future continuation of the codebase.

A.1 APPENDIX PURPOSE

The main chapters cover the report view. This appendix keeps the practical details, debug notes, and next-step ideas in one place.

A.2 COMPLETE BUILD AND RUN WORKFLOW

The project uses plain GCC and does not depend on external networking frameworks. Each version follows the same basic run order.

A.2.1 Compile

For version N:

1. `gcc -std=c99 -o receiver_N receiver_N.c`
2. `gcc -std=c99 -o sender_N sender_N.c`

A.2.2 Execute

1. Open terminal 1 and run `./receiver_N`
2. Open terminal 2 and run `./sender_N`
3. Type the test message in the sender terminal.

A.2.3 Operational Notes

1. Start the receiver first so the bind path is ready.
2. Use only one sender and receiver pair on port 9090 at a time.
3. If a process hangs, stop the old binaries and run again.

A.3 DEBUG CHECKLIST USED IN THIS PROJECT

The following checklist was used repeatedly and is useful for future maintenance.

1. Check the sender address and port.
2. Check that the receiver binds cleanly and handles errors.
3. Make sure packet fields are set before checksum work begins.
4. Make sure checksum recomputation starts from a zero checksum field.
5. Make sure sender base and next_to_send stay in order.
6. Make sure ACK handling moves the window only once.
7. Make sure duplicate packets are not delivered twice.
8. Make sure the timeout is reasonable and retries cannot loop forever.
9. Make sure the FIN exchange finishes and sockets close cleanly.

A.4 DETAILED BEHAVIOR: SENDER CORE LOOP

Across advanced versions, sender logic follows a common event-driven pattern:

1. Set up the socket, destination, and timeout values.
2. Do the handshake when the level needs it.
3. Split the message into packets or chunks.
4. Send packets while keeping the in-flight count inside the window.
5. Wait for ACKs and move the window forward.
6. Watch for timeout or duplicate ACK events and retransmit when needed.
7. Close the connection with FIN logic.

A.4.1 State Variables

The most influential sender variables in high-level versions are:

1. `base`: the oldest packet still waiting for ACK.
2. `next_to_send`: the next packet ready to go out.
3. `window_size`: the current send limit.
4. `rto_ms`: the retransmission timeout in milliseconds.
5. `cwnd` and `ssthresh`: the congestion-control values.
6. duplicate ACK counters for each ACK number.

A.5 DETAILED BEHAVIOR: RECEIVER CORE LOOP

Advanced receiver implementations use predictable stages:

1. Check the packet checksum and flags.
2. Compare the sequence with `expected_seq`.
3. Deliver in-order data right away and move forward.
4. Buffer out-of-order data that still fits in the window, then ACK it.
5. Re-ACK duplicates so the sender can recover.
6. Handle FIN and free resources cleanly.

A.5.1 Receiver Buffering in Selective-Repeat Style

The selective-repeat style receiver uses parallel arrays for packets, lengths, and presence. That keeps reordered data safe and lets delivery resume in the right order once the missing packet arrives.

A.6 VERSION-BY-VERSION PITFALL NOTES

This section captures practical issues found during progressive development.

A.6.1 v0–v2 Pitfalls

1. Bit order mistakes can rebuild the wrong character.
2. Missing null terminator logic can leave the receiver waiting.
3. `usleep` timing depends on the system.

A.6.2 v3–v7 Pitfalls

1. Parity checks need the same rules on both sides.
2. ACK/NAK formats must be clear or the sender can stall.
3. Retry loops need a limit so they do not run forever.

A.6.3 v8–v10 Pitfalls

1. Handshake flags must not clash.
2. Recompute the checksum only after all payload bytes are set.
3. Sliding-window pointer math can easily slip by one.

A.6.4 v11–v14 Pitfalls

1. Out-of-order flush must keep the sequence intact.
2. Adaptive RTO can swing too much without steady RTT samples.
3. Duplicate ACK counters must reset after each state change.
4. Fast recovery can go wrong if ACKs are read too loosely.

A.7 EXPERIMENT DESIGN TEMPLATE

A repeatable experiment protocol is essential for fair comparison. The template used in this project is shown below.

Table A.1: Standard experiment template used for each protocol version.

Field	**Description**
Version	sender_N / receiver_N pair
Message length	Short (20 chars), medium (60 chars), long (95 chars)
Fault setting	No fault / delay / simulated loss / duplicate packet scenario
Runs per case	Minimum 10 runs for stable average
Metrics	Completion time, retransmissions, ACK count, success/failure
Notes	Any abnormal behavior or timeout event logs

A.8 SAMPLE OBSERVATION NARRATIVE

The following narrative style was used to explain the test runs.

> In level 9, with a moderate induced delay, the sender kept timing out at the window base. Retransmission from base brought progress back, but the overall time to completion was much longer than in a run without delays. The receiver's ACK pattern confirmed the overall behavior.
>
> In level 12, with a variable delay, adaptive RTO cut down on false timeout events after the first RTT stabilized. The retransmission ratio went down when compared to the fixed-timeout setup.
>
> In level 14, with a fake packet gap, the triple duplicate ACK condition caused fast retransmit to happen before the timeout ran out. Recovery was clearly faster than the path that was based on timeouts.

A.9 ILLUSTRATIVE STATE MACHINE VIEW

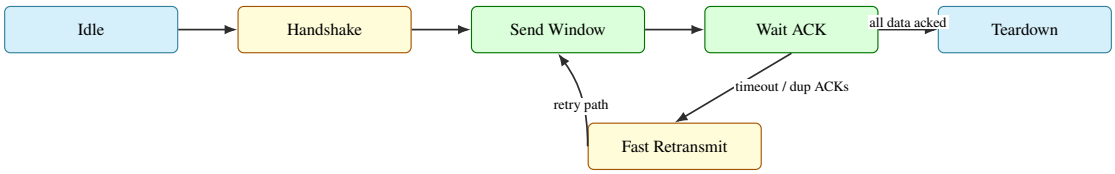

Figure A.1: Simplified sender-side operational state flow in advanced versions.

A.10 PROTOCOL TRACE FORMATTING RECOMMENDATION

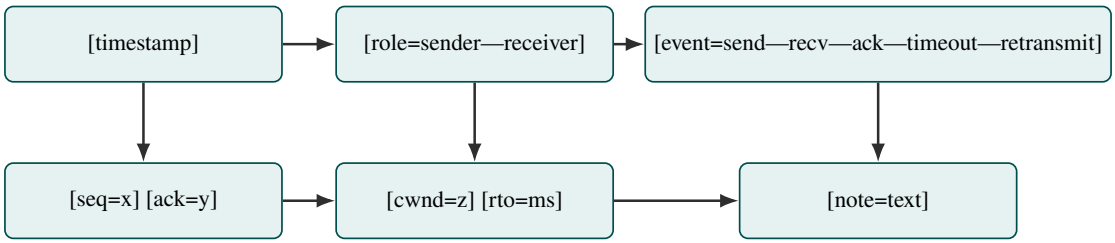

Figure A.2: Recommended structured protocol-trace fields for automated analysis.

A.11 SUGGESTED QUANTITATIVE EXTENSIONS

Future project groups can convert this implementation into a mini benchmark suite by adding automated scripts. Useful extensions include:

1. throughput versus message size for each version,
2. retransmission ratio versus injected packet loss,
3. completion time under delay jitter,
4. fairness with two parallel sender flows,
5. adaptive RTO sensitivity to α and β.

A.12 SECURITY AND RELIABILITY PERSPECTIVE

The project intentionally focuses on reliability and congestion behavior, not security. But in real-world use, integrity and authenticity still matter. Checksum catches random corruption, but it does not stop a malicious change. A production extension would include:

1. message authentication,
2. optional encryption,
3. replay protection,
4. session identity checks.

These sit outside the transport logic, but they matter in real systems.

A.13 CODE ORGANIZATION RECOMMENDATIONS

As code complexity grows, modularization becomes important. Recommended refactoring pattern:

1. `packet.h/.c` for packet structs and checksum helpers,
2. `timer.h/.c` for RTT, RTO, and timestamp helpers,
3. `window.h/.c` for send and receive window work,
4. `congestion.h/.c` for cwnd and recovery steps,
5. `main_sender.c` and `main_receiver.c` for the main flow.

This separation reduces duplication across version files.

A.14 REPRODUCIBILITY NOTES

To ensure reproducibility of results in academic evaluation:

1. note the compiler version and build flags,
2. note the operating system and kernel details,
3. close other network-heavy applications,
4. run each experiment several times,
5. report the average and spread (minimum/maximum or standard deviation).

Without reproducibility controls, protocol comparisons may appear inconsistent.

A.15 EXPANDED TESTING SCENARIOS AND INTERPRETATION

Some suggested scenarios are clean-channel baseline, delayed ACK, and packet loss. injection, messages coming in out of order, duplicate ACK bursts, and messages that are almost too big tests. These cases show correctness, recovery speed, timeout behavior, and quality of congestion adaptation.

A.16 EVALUATION RUBRIC FOR ACADEMIC ASSESSMENT

The following rubric can be used during project review.

Table A.2: Suggested assessment rubric for implementation quality.

Criterion	**Weight**	**Evaluation Focus**
Correctness	30%	Data integrity checks, ordered delivery, clean teardown
Reliability	20%	Behavior under loss, timeout and retransmission policy quality
Efficiency	15%	Throughput trend improvement from low to high levels
Code Quality	15%	Clarity, modularity, safety checks, variable naming
Analysis	10%	Metric reporting and explanation quality
Documentation	10%	Diagram quality, chapter structure, reproducibility notes

A.17 FINAL REFLECTION

From an educational standpoint, the strongest part of this project is the clear link between theory and practice. Ideas from networking class turn into behaviors you can see in terminal traces. The staged versions work like a small course in protocol engineering, covering encoding, integrity checks, reliability loops, pipelining, adaptation, and congestion response.

This project also shows that protocol design is about trade-offs. Better reliability adds state. Higher throughput adds complexity. Faster recovery needs careful event handling. Better adaptability needs steady measurement. The main skill learned here is how to balance those trade-offs.

A.18 APPENDIX SUMMARY

This appendix adds practical details, implementation tips, and ready-to-use steps to the main report. Together with the main chapters, it tells the full story from basic UDP communication to TCP-like reliability and congestion control behavior in C. It is meant as a simple transport-control learning aid.

BIBLIOGRAPHY

Allman, M., **V. Paxson**, and **E. Blanton** (2009). Tcp congestion control. RFC 5681. URL https://www.rfc-editor.org/rfc/rfc5681.

Borman, D., **B. Braden**, and **V. Jacobson** (2014). Tcp extensions for high performance. RFC 7323. URL https://www.rfc-editor.org/rfc/rfc7323.

Hall, B. B. (2024). Beej's guide to network programming. Online guide. URL https://beej.us/guide/bgnet/.

Kurose, J. F. and **K. W. Ross**, *Computer Networking: A Top-Down Approach*. Pearson, 2021, 8 edition. ISBN 9780136681557.

Mathis, M., **J. Mahdavi**, **S. Floyd**, and **A. Romanow** (1996). Tcp selective acknowledgment options. RFC 2018. URL https://www.rfc-editor.org/rfc/rfc2018.

Paxson, V., **M. Allman**, **J. Chu**, and **M. Sargent** (2011). Computing tcp's retransmission timer. RFC 6298. URL https://www.rfc-editor.org/rfc/rfc6298.

Postel, J. (1980). User datagram protocol. RFC 768. URL https://www.rfc-editor.org/rfc/rfc768.

Postel, J. (1981). Transmission control protocol. RFC 793. URL https://www.rfc-editor.org/rfc/rfc793.

Stevens, W. R. and **G. R. Wright**, *TCP/IP Illustrated, Volume 2: The Implementation*. Addison-Wesley, 1995. ISBN 9780201633542.

Tanenbaum, A. S. and **D. J. Wetherall**, *Computer Networks*. Pearson, 2011, 5 edition. ISBN 9780132126953.